

Navya — Indic-CodecFake

RTX 4060 · you own the matched real/fake Indic data and every cross-dataset number

Your job

MLAAD Indic — end to end, then re-measure everything

You own one of the two new spoof datasets, start to finish: download, extract, label, verify, hand Yugal a finished embedding root. Then you re-run every cross-dataset number against the new head, because nothing on a slide may come from a model we are no longer shipping.

#	JOB	ROUGH TIME
1	MLAAD Indic → <code>outputs\embeddings_mlaad_indic</code>	~1h download + 1h GPU
2	Re-measure DF21 + Indic holdout on Yugal's new head	~30 min
3	Email the Indic-CodecFake authors	5 min
4	Back up your embeddings to an external drive	background

Why MLaAD is the one that matters

It is up to **140 different TTS models** across 51 languages, and its Indic slice is about 45 hours: Hindi 15.1h, Bangla 14.1h, Tamil 5.0h, Kannada 4.2h, Marathi 3.0h, Urdu 2.1h, Malayalam 1.8h. It comes from Müller et al. — the same group behind In-the-Wild, which we already cite on slide 3.

The 140 models are the point, not the hours. A head trained on one generator learns *that generator*. That is exactly how we ended up with a detector that only understands 2019-era English TTS.

Step 1

Setup

```
cd <your Sonix folder>
.\.venv\Scripts\Activate.ps1

git fetch origin
git checkout yugal/ml-pipeline
git pull
pip install -U huggingface_hub
```

That pull is required — `prep_mlaad.py` is new and does not exist on your branch. Check:

```
Test-Path prep_mlaad.py, src\extract_embeddings.py, stamp_labels.py
```

All three must be `True`. If `prep_mlaad.py` is `False` you did not pull.

Step 2

Download — start this first, it runs unattended

```
python prep_mlaad.py --list
```

Prints the language folders that actually exist in the repo. Then:

```
python prep_mlaad.py --langs hi,bn,ta,mr,kn,ml,ur `
--out data\mlaad_indic --per-model 150
```

DO NOT DROP --PER-MODEL

It caps how many clips any single TTS model contributes. Without it, one prolific model supplies most of the class and the head learns to detect *that model* rather than synthesis — a new version of the exact bug we are fixing. 150 per model is the target.

The full repo is 183 GB; this pulls only the seven language folders you named, roughly 15 GB. It resumes — if it drops, re-run the identical command.

GATE 1 – BEFORE YOU SPEND GPU TIME

The script prints how many (language, model) pairs it found and the clip count. You want **several thousand clips across all seven languages and many models**. If it found one language, or one model, stop and tell Yugal — extracting a lopsided set wastes an hour and produces data we cannot use.

Step 3

Extract embeddings (GPU, ~1 hour)

```
python src\extract_embeddings.py --split train --audio-dir data\mlaad_indic `
--out outputs\embeddings_mlaad_indic --batch 8
```

CUDA out of memory → `--batch 4`, then `--batch 2`. Progress check, any time:

```
(Get-ChildItem outputs\embeddings_mlaad_indic\train\shard_*.npy |
Where-Object {$_.Name -notlike "*labels*"}).Count
```

Step 4

Stamp the labels — the step that must not be skipped

`--audio-dir` extraction marks **every row label 0 (real)** by default. These are all fakes. Stamp them:

```
python stamp_labels.py --emb-dir outputs\embeddings_mlaad_indic\train --label 1
python stamp_labels.py --emb-dir outputs\embeddings_mlaad_indic\train --check
```

GATE 2 – READ THE CHECK OUTPUT, DO NOT ASSUME

It must report **every row as spoof (1)**. If any bonafide rows remain, stop. Training on unstamped fakes teaches the model that synthetic Hindi is genuine — strictly worse than not having the data at all.

Note: your own fix to `stamp_labels.py` is what makes this work. Before 2 Sept it never successfully wrote a label.

Then send Yugal

- The folder `outputs\embeddings_mlaad_indic\`

- The full `--check` output, pasted
- The `(language, model)` summary `prep_mlaad.py` printed

Step 5

Re-measure everything on the new head

Once Yugal ships `head_v3.pt`, every cross-dataset number has to be re-derived. The old figures came from models we are no longer shipping.

```
python src\eval.py --split eval --emb-root outputs\embeddings_df21 `
  --model-ckpt outputs\models\head_v3.pt --out-scores outputs\scores_df21_v3
python src\metrics.py --split eval --scores-dir outputs\scores_df21_v3 `
  --out-dir outputs\plots_df21_v3
```

Then the Indic holdout, with your own tool:

```
python src\eval.py --split train --emb-root outputs\embeddings_indicvoices_ho `
  --model-ckpt outputs\models\head_v3.pt --out-scores outputs\scores_ho_v3
python flag_rate.py --split train --scores-dir outputs\scores_ho_v3
```

SAY PRECISELY WHAT “FLAGGED” MEANS

`flag_rate.py` counts one score per 4-second chunk, no smoothing, no hysteresis. So the figure is “X% of genuine 4-second chunks”, never “X% of calls” — the demo path applies a 5-window moving average and alarms less often. Conservative is good; imprecise phrasing is what a follow-up question kills.

Also: a **second seed** on the final head, 15 minutes. It turns “5.27%” into a measurement with error bars rather than one lucky number. Until then we say “≈5.3%”. And keep quoting **75% coverage** on every DF21 figure.

Step 6

Two small things worth doing

- **Email the Indic-CodecFake authors.** Their paper is published but the dataset link on helixometry.github.io/IndicFake is still a placeholder — only the codec code is up. It would be the best dataset for this problem (one-to-one real/fake pairs of the same Indian speaker across 8 neural codecs), so it is worth asking. Do not wait on a reply.
- **Back up your embeddings.** DF21 and IndicVoices exist on exactly one machine. If it dies, every cross-dataset number in the deck becomes unreproducible. An external drive is enough.
- **Retire your `metrics.py`** in favour of Yukti's — hers has `--scores-file` and log-scale DET axes. Two versions in circulation is how two slides end up disagreeing.

Credit

What your work already established

Your recording-level holdout split is the reason we can trust any Indic number as generalisation rather than memorisation — it groups by recording id, then recomputes the intersection and *refuses to write* if any recording appears on both sides. And you found that `stamp_labels.py` had never once written a label successfully, which was silently corrupting anything anyone stamped.

The IndicVoices result was not wrong. It measured a real false-alarm problem and fixed it. It simply could not catch the second bug, because no Indian spoofs existed to test against. That is what this round adds.

Troubleshooting

MESSAGE	FIX
can't open file 'prep_mlaad.py'	You did not pull, or wrong folder
download stops partway	Re-run the identical command — it resumes
none of the requested languages exist	Run <code>--list</code> and use the real folder names
CUDA out of memory	<code>--batch 4</code> , then <code>--batch 2</code>
check shows bonafide rows	Stop. Re-run the stamp; if it persists, tell Yugal
Activate.ps1 cannot be loaded	<code>Set-ExecutionPolicy -Scope Process -Bypass</code>

The situation

What broke, and why

Our detector fails on Indian speech, and we now know exactly why.

Every model up to `head_robust_v2` was trained on ASVspoof 2019 — English studio speech — plus augmentations that vary *rooms*, *noise* and *phone channels*. None of them vary *language*. So on 42,082 genuine IndicVoices clips, the baseline called **56% fake** and `robust_v2` called **58.6% fake**. That is why the Modi and Bachchan clips failed.

Navya then added IndicVoices to training and Indian false alarms collapsed to **0.02%**. Real fix? No — a second bug. IndicVoices is **bonafide-only**: every one of its ~37,000 rows is labelled real. So the only thing the model could learn from Indian audio was “*Indian phonetics* → *real*”. Tested on a cloned Amitabh Bachchan clip, `head_full_ho` calls it **real**.

MODEL	REAL BACHCHAN	CLONED BACHCHAN	WHAT IT ACTUALLY LEARNED
head_robust_v2	fake ✗	fake ✓	“Indian audio = fake”
head_full_ho	real ✓	real ✗	“Indian audio = real”

Neither is detecting anything. Each gets one column right by accident.

THE ROOT CAUSE, IN ONE SENTENCE

As long as every Indian-language sample in training is labelled real, the language itself is a shortcut to “real”, and a cloned Indian voice walks straight through. No threshold, augmentation or calibration fixes this.

The fix

Put Indian languages on **both sides of the label**, from **several different synthesis families**. Then the model cannot use language or accent as a cue — it has to find real synthesis artefacts or nothing.

DATASET	SYNTHESIS FAMILY	OWNER
Indic-CodecFake	Neural audio codecs — 8 of them (DAC, Encodec, SoundStream, SpeechTokenizer, FunCodec, AudioDec, SNAC, MIMI)	Navya
MLAAD (Indic slice)	Text-to-speech — up to 140 models	Akshat
IndicSynth	Voice conversion (freevc24)	Akshat
IndicVoices <code>have it</code>	Real human speech (bonafide side)	Navya

Three *families*, not just more hours. A head trained on one generator learns that generator — which is precisely the trap we fell into with English-only ASVspoof 2019. Generator diversity is what buys generalisation.

How we train

One model, many ablations

Two questions came up, so here they are answered for everyone.

Is the new model built from scratch, or on top of the latest one?

Neither, exactly — and this matters. Our pipeline has two stages:

STAGE	COST	REUSABLE?
audio → embeddings (frozen wav2vec2 XLS-R, 300M params)	GPU, hours	Yes — forever, per dataset
embeddings → head (MLP, 262,657 params)	CPU/GPU, minutes	Cheap to redo

So we never “continue training” `head_full_ho`. We retrain a **fresh head on the union of every embedding root** — the old ones plus the new ones. That is what `--extra-emb-root` does, and it stacks (it uses `action="append"`, verified).

Fine-tuning the existing head on only the new data would be actively wrong: it would forget the English attacks it can currently catch.

One model or several?

One production model, plus four throwaway ablation heads for the results slide. Once the embeddings exist, each head takes minutes, so the ablations are nearly free — and they are what let us say “*we measured which data helped*” instead of “*we threw everything in and hoped*”.

The expensive, parallel part is extraction. That is why it is split three ways across our GPUs. Training is serial and Yugal does it.

THE RULE THAT MAKES OR BREAKS THIS

Cap clips per (language × generator). If one TTS model contributes 40,000 clips and another contributes 200, the head learns to detect the loud one. Every extraction step below has a cap flag. Use it.

What must never enter training

These are test sets. Training on any of them destroys the numbers that prove we generalise, and a judge who spotted it would be right to dismiss the project.

- **ASVspoof 2021 DF** — our cross-dataset headline
- **In-the-Wild** — the honest generalisation test, named on slide 3
- **The IndicVoices holdout** (`embeddings_indicvoices_ho`) — 12% held out by recording
- **Akshat's 20 labelled clips** and **the Bachchan pair** — our real-world evidence

Everything else — ASVspoof 2019 train, the augmented copies, IndicVoices train split, Indic-CodecFake, MLAAD, IndicSynth — is fair training data.

SECOND HARD RULE – LABEL STAMPING

`extract_embeddings.py --audio-dir` marks **everything label 0 (real)** by default. Every spoof folder must be stamped `--label 1` afterwards, and verified with `--check`. Skip it and your fakes train as genuine — a silent, catastrophic bug that nothing warns you about.

`stamp_labels.py` was broken until 2 Sept (it never wrote a label successfully). Make sure you have pulled since then, and always read the `--check` output rather than assuming.

Sections after your own tasks are identical in all three handbooks, so nobody has to read someone else's. Datasets: Indic-CodecFake (helixometry.github.io/IndicFake) · MLAAD (deepfake-total.com/mlaad) · IndicSynth (huggingface.co/datasets/vdivyasharma/IndicSynth). When in doubt, stop and ask Yugal — a wrong number is far worse than a missing one.